

## **Overview:**

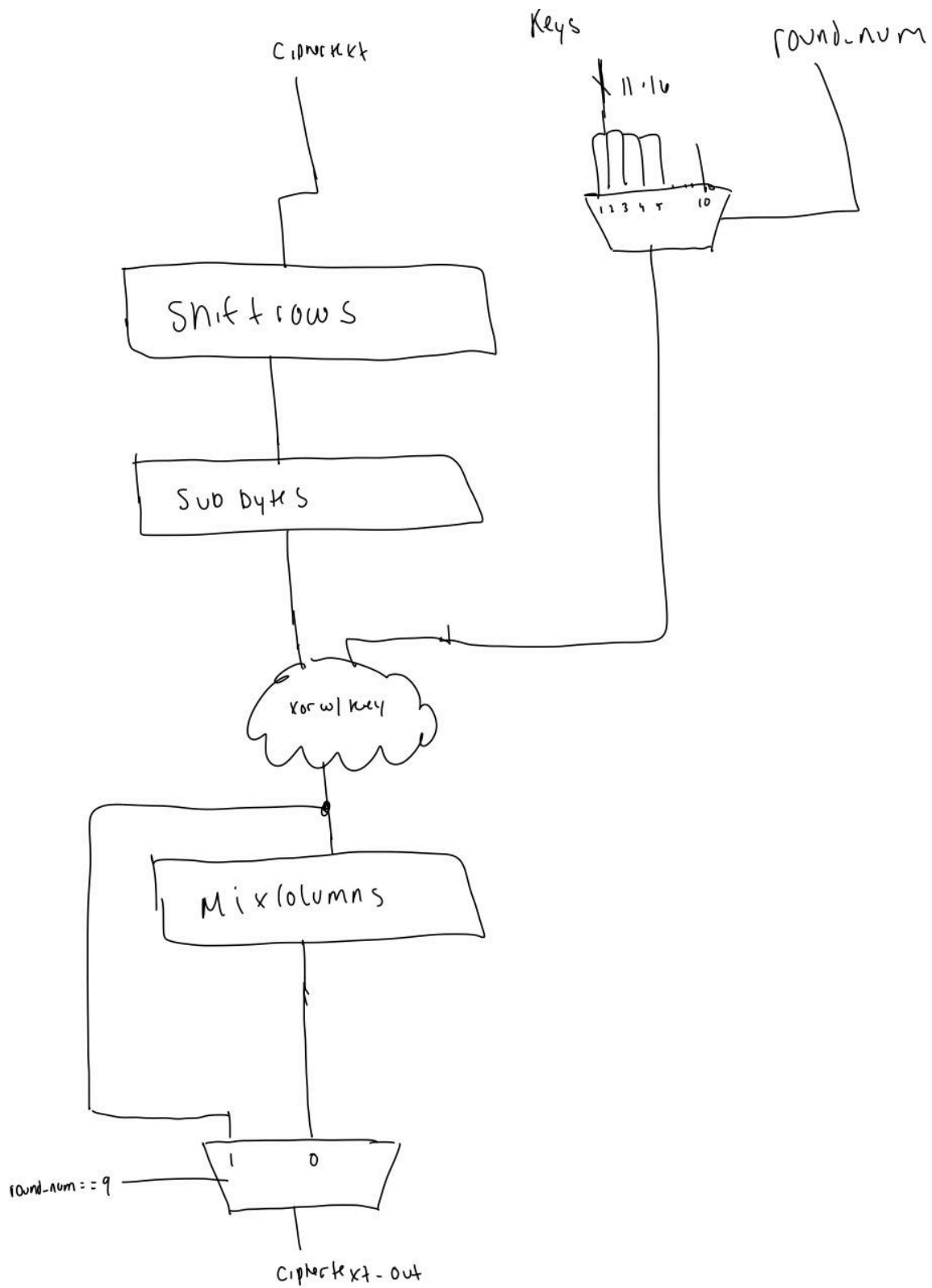
At this stage, I am choosing to make a “mini” AES implementation that will take in a 16 bit cipher text, and a 16 bit key and output the correct 16 bit plaintext. In order to minimize the area of my final product, I am planning on pipelining my implementation, meaning I will not create a module for each round, rather the decryption itself will take 10-cycles.

## **AES Modules:**

My design will consist of the modules: InvShiftRows, InvSubBytes & InvSbox, Sbox, InvMixColumns and Round\_Scheduler. The InvSubBytes module substitutes the bytes seen in the ciphertext depending on the appropriate output from the InvSbox. In this simplified version of AES, the sbox will function as an inverter. The ShiftRows module will perform a bit shift of each data row (instead of a byte shift as seen in a typical AES implementation), depending on the index. The InvMixColumns module will multiply the upper 8 bits by 9, and the lower 8 bits by 14 using operations in a Galois field. And the round scheduler will perform operations on the input key, to determine what the next keys for each round should be. .

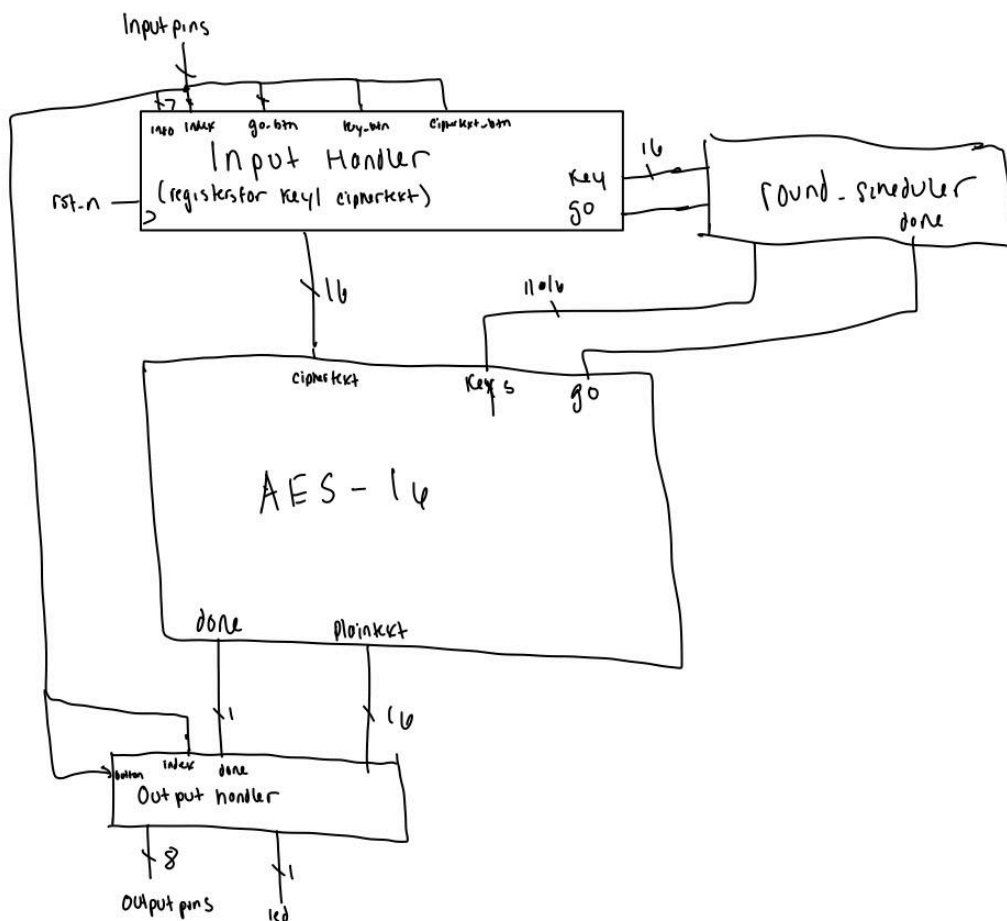
## **Overall Design layout:**

This picture shows how each of these modules will be connected at a high level to complete one round of decryption. Note that as of right now, I am planning for this stage to be fully combinational. However this might change depending on area restrictions.



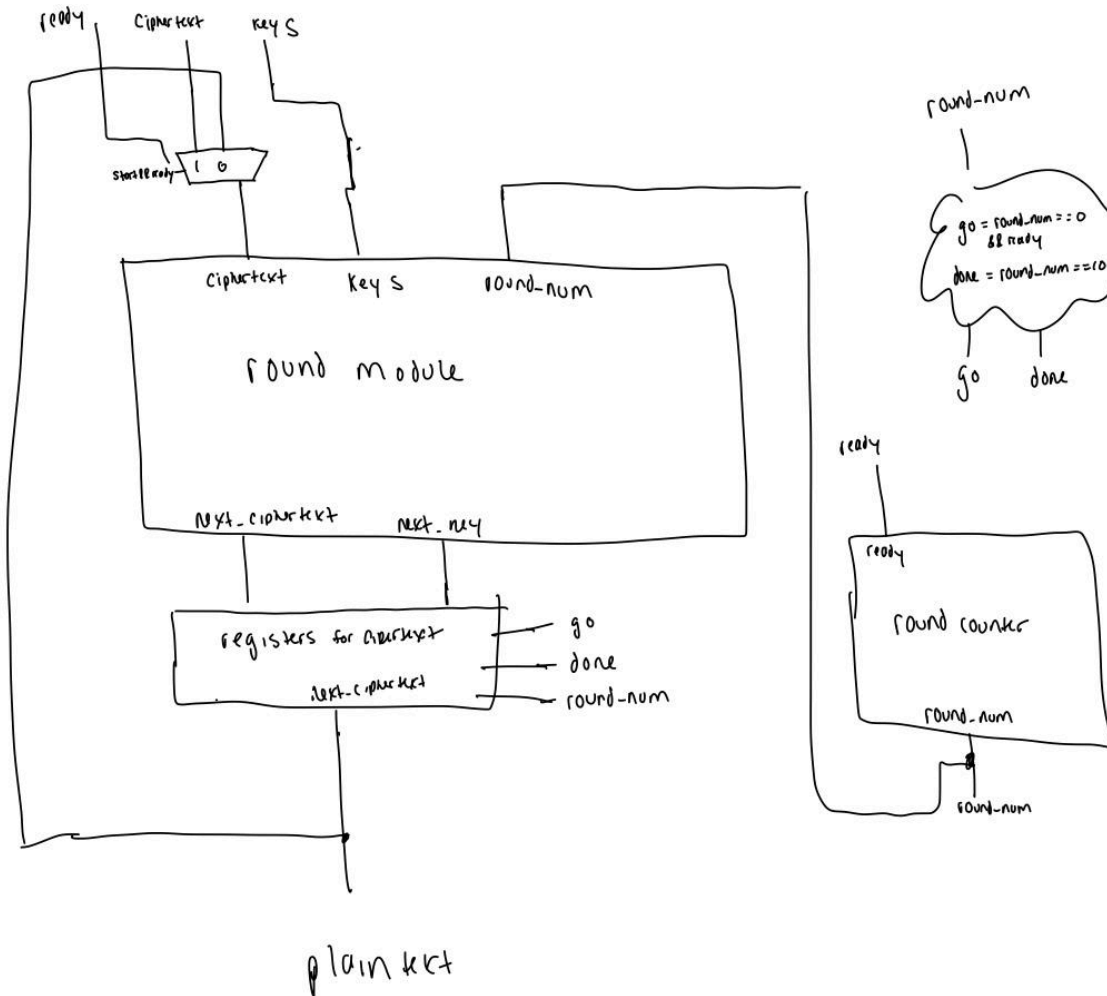
## Top Level Design:

The current top level block diagram is shown below, once the input is ready, the decryption will begin from the first round, and continue until the counter reaches 10, indicating that decryption has finished. Once this happens, the ready signal will be asserted and this will indicate that the values at the output pins are correct. As of right now, to reduce logic, I am planning on using the chip's reset each time the user needs to compute a new value, but this might change in the future to be another on-chip button.



The image below shows the AES module in more detail. The counter functions like a control FSM. It starts at 0, and increments each cycle until it reaches 10. When the counter is equal to 10, the done signal is asserted. When ready is high and the counter is 0, the go signal will be

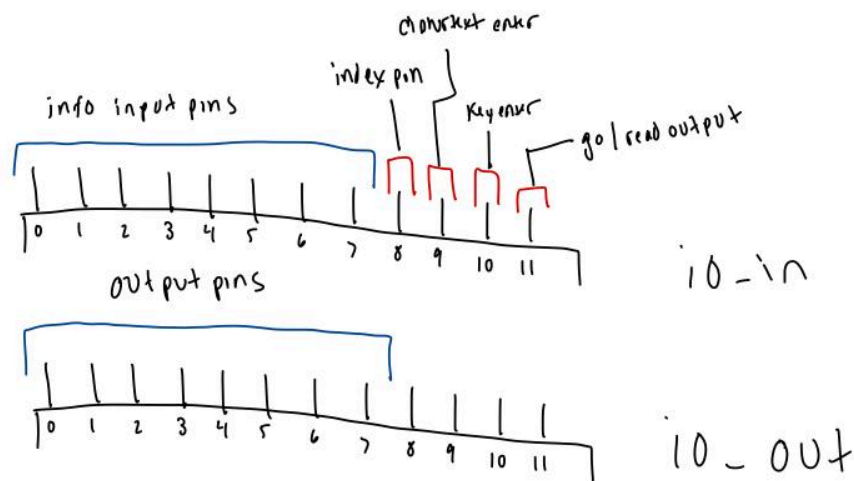
asserted. This is used to feed the original inputs through the round module rather than the pipelined inputs.



### Handling Inputs/Outputs:

The input/output modules shown in the top level design are left vague on purpose, as this is something that I have not completely worked out yet. Initially I was planning on taking in bare-metal inputs in a way that is similar to a scan chain: the user would put the values of the key and ciphertext (from MSB to LSB) on 8 of the input pins, then press the button to indicate that the value is correct, then I would shift them.

However, this seems like a method that would be prone to errors. Instead I was thinking of using another approach shown below, where the user can choose if they are inputting a byte for the key or the ciphertext, and can also overwrite previous inputs if there is a mistake.



io-in :

- 7:0 → information pins: key / ciphertext input here in 8-bit chunks
- 8: → index pin → selects upper chunk or lower chunk to input/output  
when 1: 15:8, when 0, 7:0
- 9: → is high when entering a ciphertext value
- 10 → is high when entering a key value
- 11 → is high when starting aes → is finished is high when printing output

## Testing:

To test this project I was planning on creating test vectors by hand. To my understanding, there should not be many edge cases for the AES-decryption module, so my plan was to make or find a golden module implementation in C or python, and check the outputs of my module against

that. I also plan on testing each submodule individually, as I am working on them. This is much simpler however, so I think I will create these test vectors and their expected outputs by hand, and use simple assertions to check if they are as expected.

I will also need to test the modules that take in the inputs/outputs. This will be done using a basic SystemVerilog testbench, and I will use displays/simple assertions to make sure the input/output registers are correct, and that signals (such as the input ready/done signals are being asserted on the correct clock cycles).